

CS6650 Midterm Mastery

Zhuoyue Lian

Can we make distributed systems behave like single systems?

Answer: No.

- Network partitions happen
- Messages get lost or delayed
- Clocks don't synchronize perfectly
- Nodes fail independently

Fundamental challenge

- reliability and performance across multiple machines
- physics and hardware constraints prevent perfect coordination

Two key impossibility results

FLP - can't guarantee both correctness and completion with failures.

CAP - during network partitions, choose consistency or availability, not both.

CAP Theorem - Product API with ALB

Health check mechanism implements: **CP** (Consistency + Partition Tolerance)

- Health checks remove unhealthy instances
- Traffic only to healthy instances
- **Result:** Consistent data, but reduced capacity during failures (sacrifices some availability for consistency)

Alternative design: **AP** (Availability + Partition Tolerance)

- Serve from stale cache during partitions
- All instances stay available
- **Result:** Higher availability, but inconsistent data

Abstraction Layers Have Costs

Key Insight: Higher abstraction = More convenience, Higher coordination cost

Coordination costs at different levels:

This hierarchy of costs is critical - it explains why distributed systems are fundamentally slower than single-machine systems for operations requiring coordination.

Layer	Context Switch Time	Example
Goroutines (single-core)	~295 nanoseconds	Single OS thread
Goroutines (multi-core)	~328 nanoseconds	Multiple OS threads
Processes	1-10 microseconds	Different ECS tasks
Containers/Network	Milliseconds	Cross-instance calls
Network Partitions	Minutes to hours	Datacenter splits

Concurrency Lessons

Threading Experiments - choosing the right synchronization mechanism

When 50 goroutines wrote to:

Plain map (no sync): **CRASHED**

Mutex: 11.5ms

RWMutex: 13.1ms

sync.Map: **5.7ms** (fastest) - designed specifically for concurrent access

Without atomicity: Lost 12,000-18,000 increments

With atomic operations: Perfect 50,000 count

Takeaway: Specialized data structures beat manual synchronization

Information Hiding (Parnas)

Parnas's 1972 paper: Each module should hide a design decision likely to change

Wrong approach: Modules based on processing steps

Input → Process → Store → Output

Problem: Change storage format = change ALL modules

Right approach: Modules hide design decisions

Line Storage Module (hides: storage format)

Search Module (hides: algorithm details)

Benefit: Change storage = change ONE module

Real example: In the Product API, "checking 100 products per search" should be hidden so changing to 500 doesn't break other code.

Part 2 - The Crash Experiment

Problem Created: Memory leak through goroutine leaks

Buggy Code Pattern:

// Each request spawns a goroutine that NEVER exits
// Continuously allocating memory

```
go func() {  
    for {  
        leak_memory() // 5MB/second  
        sleep(1)  
    }  
}()
```

Result: Task crashes in 90-120 seconds under load

```
router.POST("/products/analyze/:id", func(c  
*gin.Context) {  
    id := c.Param("id")  
  
    // BUG: Goroutine never exits!  
    go func() {  
        for {  
            // Allocate 5MB every second and keep  
            it  
                chunk := make([]byte, 5*1024*1024)  
                leakedMemory = append(leakedMemory,  
chunk)  
                time.Sleep(1 * time.Second)  
            }  
        }()  
  
        c.JSON(200, gin.H{"message": "Analysis  
started", "product_id": id})  
    })
```

Crash Evidence

Local Testing:

- Initial: 1 MB, 3 goroutines
- After 10 requests: 4,301 MB, 13 goroutines
- After 86 seconds: 96,257 MB
- **Leak rate: ~1,100 MB/second**

AWS Testing (512 MB limit):

- 0s: 431 MB, 51 goroutines
- 60s: 9,566 MB, 255 goroutines
- 120s: 24,717 MB, 397 goroutines → **CRASH**
- Exit code 137: OutOfMemoryError

The screenshot shows the AWS Management Console interface for the Amazon Elastic Container Service (ECS). The main view is the 'Tasks' page for a cluster named 'product-service-cluster'. The tasks are listed in a table with columns for Task ID, Last status, Desired status, Health status, Created at, Started by, and Start time. All tasks are in a 'Stopped' state. A tooltip is visible over one of the tasks, showing the 'Stopped reason' as '[product-service] Exit code: 137, reason: "OutOfMemoryError: Container killed due to memory usage"'. The console also shows filters for 'Filter desired status' (set to 'Stopped') and 'Filter launch type' (set to 'Any launch type'). The left sidebar shows navigation options for Amazon Elastic Container Service, including Clusters, Namespaces, Task definitions, and Account settings. The bottom of the console shows the footer with copyright information and links to CloudShell and Feedback.

Health checks kept passing during this time - they only detected the problem after the crash occurred (shows the gap between infrastructure-level monitoring and application-level problems)

The Fix

Fix: Bounded goroutine lifetime with context timeout (Go's context package)

Key Improvements:

Timeout-based termination: Goroutines automatically exit after 5 seconds maximum

Bounded work: Each goroutine performs exactly 10 iterations then exits

Select statement: Allows goroutine to respond to cancellation signals

Reduced memory footprint: 100 KB allocations instead of 5 MB

Context cancellation: Proper cleanup with defer cancel() and defer ticker.Stop()

```
router.POST("/products/analyze/:id", func(c
*gin.Context) {
    id := c.Param("id")

    go func() {
        // Create 5-second timeout context
        ctx, cancel :=
context.WithTimeout(context.Background(),
5*time.Second)
        defer cancel()

        ticker := time.NewTicker(500 *
time.Millisecond)
        defer ticker.Stop()

        workCount := 0
        for workCount < 10 {
            select {
                case <-ctx.Done():
                    // Timeout - exit cleanly
                    return
                case <-ticker.C:
                    // Do work
                    _ = make([]byte, 100*1024) // 100KB
                    workCount++
            }
        }
        // Work complete - goroutine exits
    }()

    c.JSON(200, gin.H{"message": "Analysis
started", "product_id": id})
})
```

Fixed Version Results

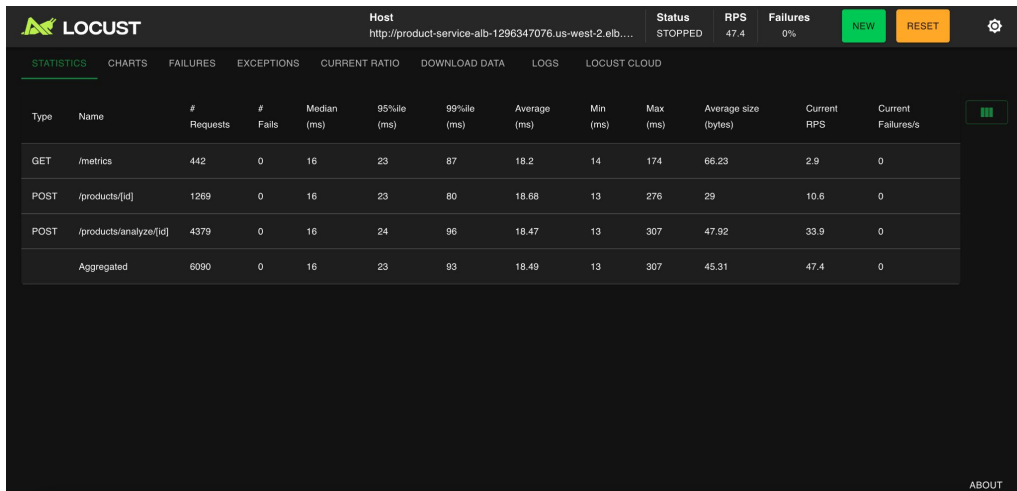
Same load test (10 users) - different behavior

Buggy version:

- Memory: 431 MB → 24,717 MB → CRASH
- Goroutines: 51 → 397 (growing)
- Runtime: 120 seconds to failure

Fixed version:

- Memory: **1-3 MB stable** (99.9% reduction)
- Goroutines: **80-95 stable** (bounded)
- Runtime: **No crash** (3+ minutes, 100% uptime)
- Request failures: **0%**



The screenshot shows the Locust web interface with a dark theme. At the top, the 'Host' is 'http://product-service-alb-1296347076.us-west-2.elb....'. The 'Status' is 'STOPPED', 'RPS' is '47.4', and 'Failures' is '0%'. There are 'NEW' and 'RESET' buttons. Below the navigation bar, the 'STATISTICS' tab is selected. A table displays the following data:

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/metrics	442	0	16	23	87	18.2	14	174	66.23	2.9	0
POST	/products/[id]	1269	0	16	23	80	18.68	13	276	29	10.6	0
POST	/products/analyze/[id]	4379	0	16	24	96	18.47	13	307	47.92	33.9	0
Aggregated		6090	0	16	23	93	18.49	13	307	45.31	47.4	0

At the bottom right, there is an 'ABOUT' link.

Key Lesson - Defense in Depth

Building reliable distributed systems requires multiple defensive layers

Infrastructure level (Health checks, auto-scaling):

- Detects failures AFTER they occur
- Cannot prevent application bugs

Application level (Timeouts, resource bounds):

- Prevents failures BEFORE they occur
- Requires explicit code design

Observability (Metrics, monitoring):

- Detects problems EARLY
- Enables proactive response

Both layers required for reliable distributed systems